Analisi e verifica di una vulnerabilità Buffer Overflow

Studente: Vincenzo Lembo
Corso: Cybersecurity & Ethical Hacking
Esercitazione: W17D4
Ambiente: Kali Linux, UTM

---

1. Introduzione

L'obiettivo dell'esercitazione è comprendere il funzionamento di una vulnerabilità di tipo **Buffer Overflow (BOF)**attraverso la creazione e l'esecuzione di un programma scritto in linguaggio C. Un Buffer Overflow si verifica quando un programma scrive una quantità di dati superiore alla capacità della memoria assegnata a un buffer, causando la sovrascrittura di aree di memoria adiacenti. Durante l'esercitazione è stato realizzato un programma volutamente vulnerabile, sul quale sono stati effettuati diversi test per osservare il comportamento della memoria e individuare eventuali errori.

---

2. Preparazione dell'ambiente

L'esercitazione è stata svolta utilizzando una macchina virtuale Kali Linux eseguita attraverso il software UTM.

Dopo aver avviato il sistema operativo, è stato aperto il terminale e raggiunta la directory Desktop mediante il comando:

```
cd ~/Desktop
```

Per verificare il percorso corrente è stato eseguito:

```
pwd
```

Il comando ha restituito:

```
/home/kali/Desktop
```



---

3. Creazione del programma vulnerabile

Attraverso l'editor di testo GNU nano è stato creato un nuovo file denominato `BOF.c` mediante il comando:

```
nano BOF.c
```

All'interno del file è stato inserito il seguente codice:

```
#include <stdio.h>


int main() {


    char buffer[10];


    printf("Si prega di inserire il nome utente:");

    scanf("%s", buffer);


    printf("Nome utente inserito: %s\n", buffer);


    return 0;

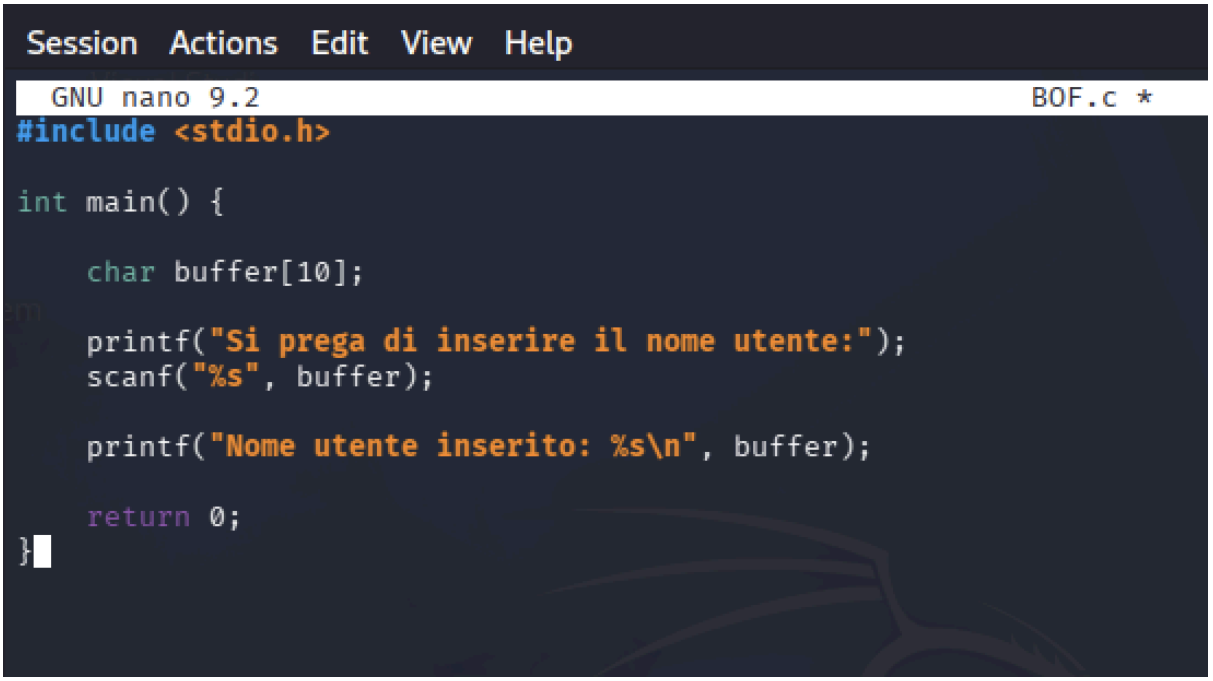
}
```

Il programma richiede l'inserimento di un nome utente e memorizza il valore nella variabile `buffer`, inizialmente dimensionata a 10 byte.

La vulnerabilità è presente nell'istruzione:

```
scanf("%s", buffer);
```

Questa istruzione non impone un limite alla lunghezza della stringa inserita, permettendo potenzialmente di sovrascrivere aree di memoria esterne al buffer.



---

4. Compilazione del programma

Dopo aver salvato il file, è stata effettuata la compilazione attraverso GCC utilizzando il comando:

```
gcc -g BOF.c -o BOF
```

Il parametro `-g` permette di includere informazioni di debug, mentre `-o BOF` specifica il nome dell'eseguibile generato.

La compilazione è terminata correttamente, senza restituire messaggi di errore.

---

5. Esecuzione e test del Buffer Overflow

5.1 Primo test con input breve

Il programma è stato avviato attraverso:

`./BOF`

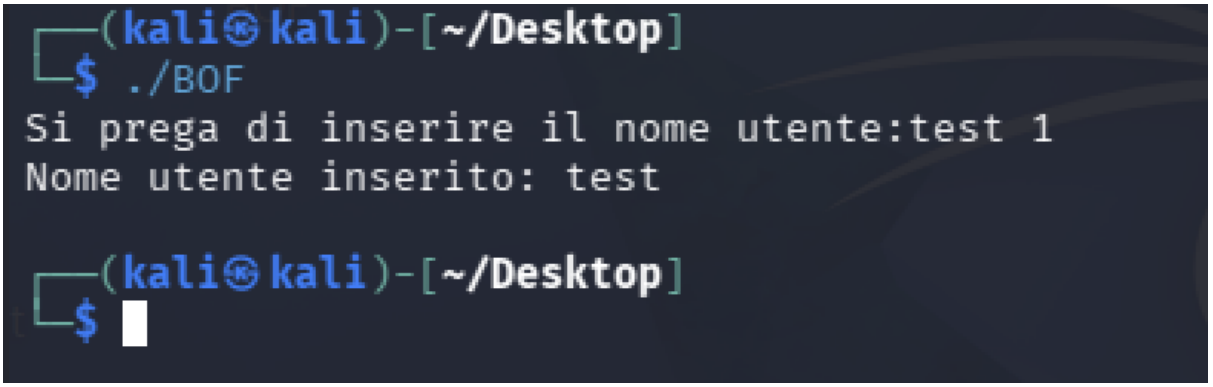
Durante il primo test è stata inserita la stringa `test 1`.

Il programma ha restituito:

`Nome utente inserito: test`

La funzione `scanf("%s", buffer)` interrompe infatti la lettura quando incontra uno spazio.

Il programma è terminato regolarmente, senza generare errori.



5.2 Secondo test con 30 caratteri

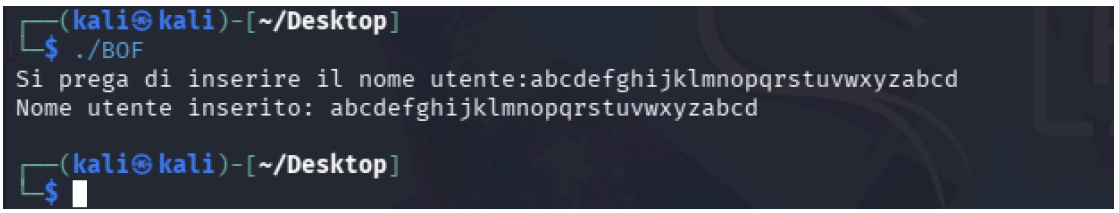
Successivamente è stato ripetuto il test inserendo una stringa di 30 caratteri:

`abcdefghijklmnopqrstuvwxyzabcd`

Il programma ha visualizzato correttamente la stringa inserita, senza generare un errore di segmentazione.

Tuttavia, il buffer disponeva soltanto di 10 byte, mentre la stringa richiedeva 31 byte, considerando anche il terminatore `\0`.

Pertanto, l'assenza di un errore visibile non dimostra l'assenza della vulnerabilità.

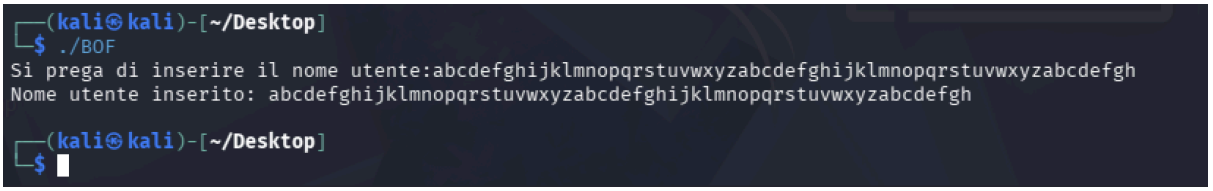


5.3 Terzo test con 60 caratteri

Per verificare ulteriormente il comportamento del programma, è stata inserita una stringa di 60 caratteri.

Anche in questo caso il programma ha visualizzato l'input senza interrompersi.

Il risultato dimostra che una scrittura fuori dai limiti di un buffer non produce necessariamente un errore immediatamente visibile.



---

6. Analisi della vulnerabilità con AddressSanitizer

Poiché i test precedenti non avevano generato un segmentation fault, è stato utilizzato AddressSanitizer, uno strumento di diagnostica della memoria supportato da GCC.

Il programma è stato ricompilato utilizzando:

```
gcc -g -fsanitize=address -fno-omit-frame-pointer BOF.c -o BOF_ASAN
```

Successivamente è stato eseguito:

```
./BOF_ASAN
```

Inserendo nuovamente la stringa di 30 caratteri, AddressSanitizer ha rilevato la vulnerabilità.

Il terminale ha restituito:

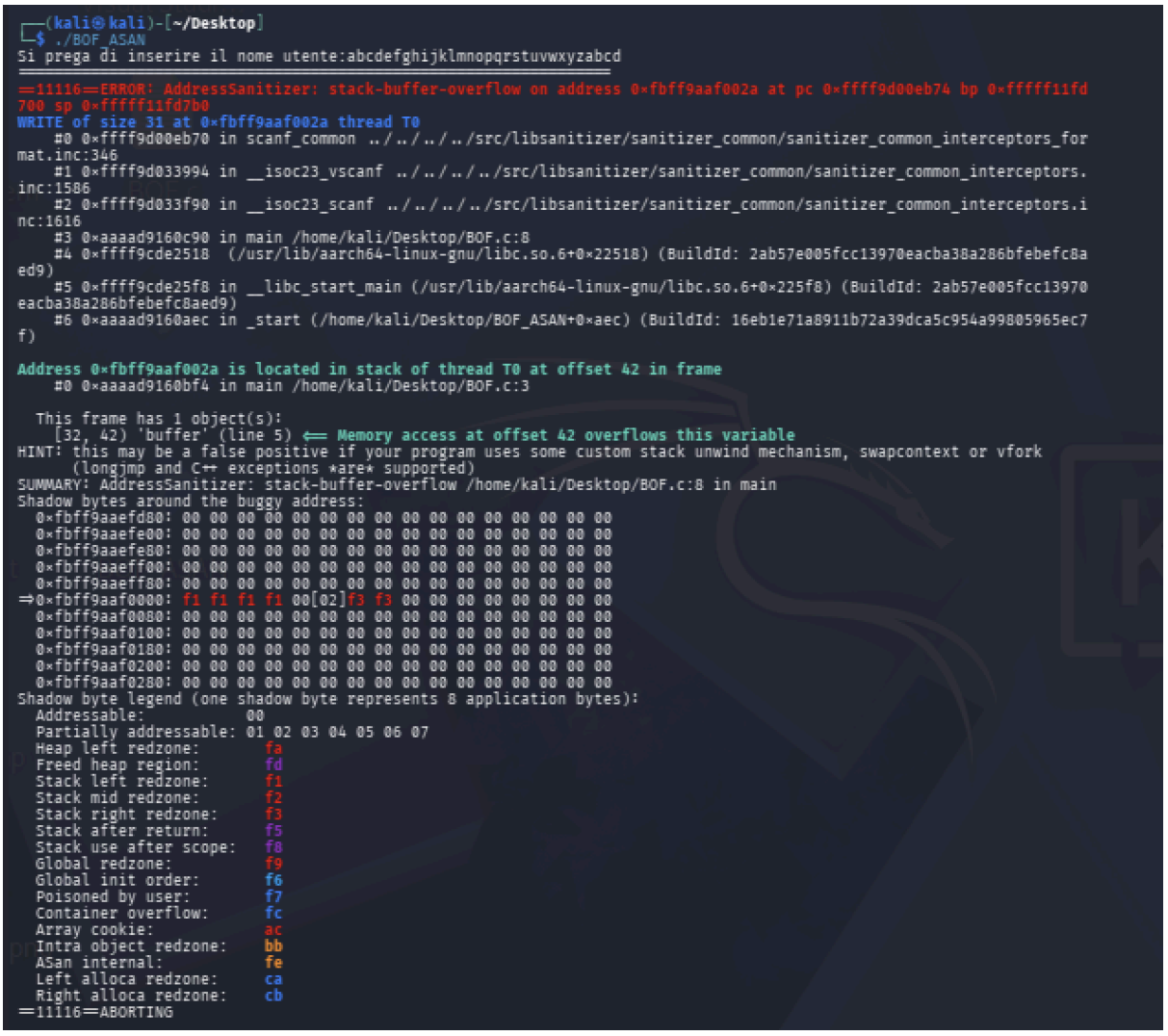
ERROR: AddressSanitizer: stack-buffer-overflow

WRITE of size 31

Il messaggio identifica una scrittura di 31 byte oltre la capacità del buffer.

L'analisi ha inoltre individuato l'istruzione responsabile dell'errore alla riga 8 del file `BOF.c`, corrispondente alla funzione `scanf()`.

Il risultato conferma la presenza di un Buffer Overflow nello stack.



7. Modifica della dimensione del buffer

Come richiesto dalla traccia, il programma è stato modificato aumentando la dimensione del buffer da 10 a 30 byte.

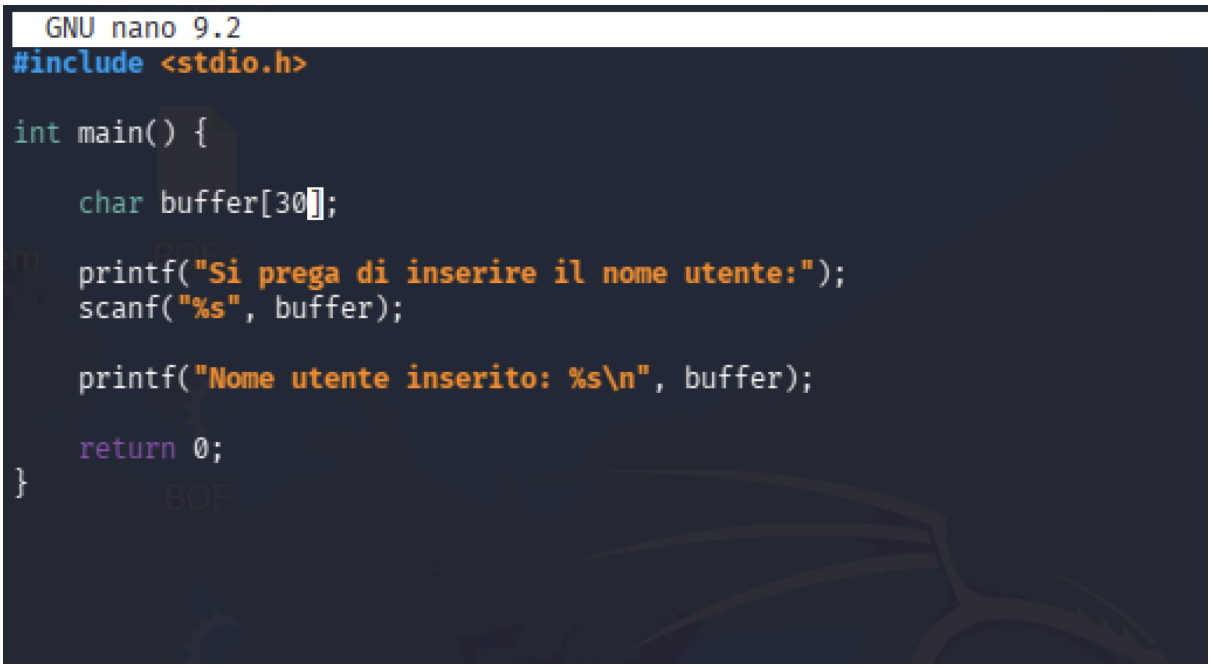
La dichiarazione originale:

```
char buffer[10];
```

È stata sostituita con:

```
char buffer[30];
```

La funzione `scanf()` è stata mantenuta invariata.



7.1 Ricompilazione e verifica

Dopo aver salvato il codice modificato, il programma è stato ricompilato:

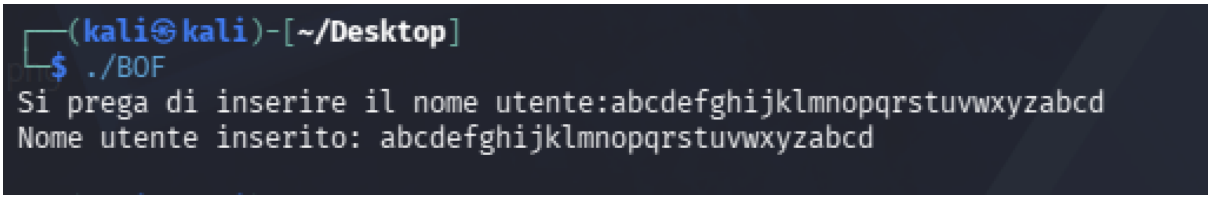
```
gcc -g BOF.c -o BOF
```

Successivamente è stato eseguito:

```
./BOF
```

Inserendo nuovamente 30 caratteri, il programma ha visualizzato correttamente la stringa senza mostrare errori.

Tuttavia, la modifica non garantisce la sicurezza del programma, poiché la funzione `scanf()` continua a non imporre alcun limite alla lunghezza dell'input.



---

8. Verifica finale con AddressSanitizer

Per accertare se la vulnerabilità fosse ancora presente, il programma modificato è stato nuovamente compilato con AddressSanitizer:

`gcc -g -fsanitize=address -fno-omit-frame-pointer BOF.c -o BOF_ASAN`

L'eseguibile è stato avviato mediante:

`./BOF_ASAN`

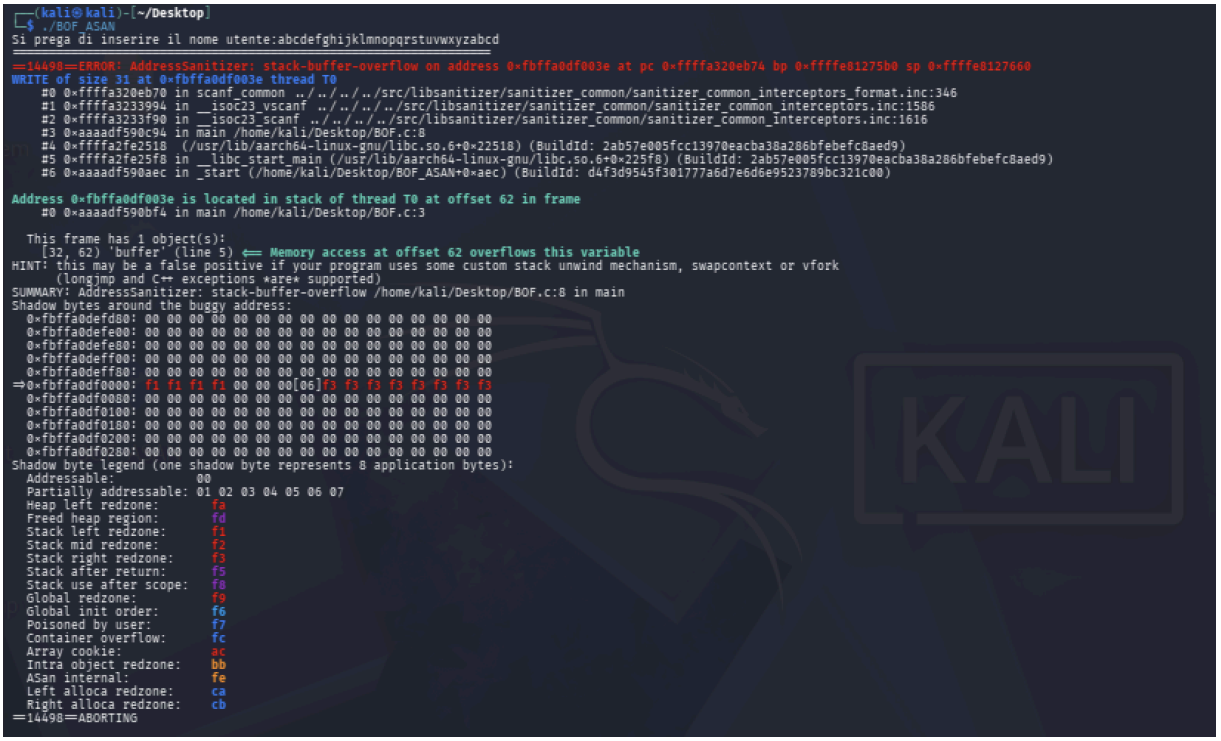
Inserendo la stessa stringa di 30 caratteri, AddressSanitizer ha restituito nuovamente:

`ERROR: AddressSanitizer: stack-buffer-overflow`

`WRITE of size 31`

Il risultato dimostra che l'aumento della dimensione del buffer non ha eliminato la vulnerabilità.

Infatti, per memorizzare 30 caratteri in una stringa C sono necessari 31 byte, considerando il carattere terminatore `\0`.



---

9. Risposta alla domanda finale della traccia

Domanda: Aumentare la dimensione del vettore a 30 basta a eliminare la possibilità di generare un Buffer Overflow?

No. Aumentare la dimensione del buffer da 10 a 30 byte non è sufficiente per eliminare la vulnerabilità.

Il problema principale risiede nell'utilizzo della funzione `scanf("%s", buffer)`, che non limita il numero di caratteri acquisiti.

Anche con un buffer da 30 byte, l'inserimento di una stringa di 30 caratteri provoca una scrittura oltre i limiti, poiché è necessario memorizzare anche il terminatore `\0`.

Una possibile soluzione consiste nel limitare esplicitamente la lunghezza dell'input attraverso:

```
scanf("%29s", buffer);
```

In questo modo vengono acquisiti al massimo 29 caratteri, lasciando un byte disponibile per il terminatore della stringa.

---

10. Conclusioni

L'esercitazione ha permesso di comprendere il funzionamento di una vulnerabilità Buffer Overflow attraverso la realizzazione di un programma C volutamente vulnerabile. Durante i test iniziali, il programma non ha generato errori visibili, nonostante l'inserimento di stringhe superiori alla capacità del buffer. L'utilizzo di AddressSanitizer ha consentito di identificare concretamente la scrittura fuori dai limiti della memoria. Successivamente, aumentando la dimensione del buffer da 10 a 30 byte, è stato dimostrato che la vulnerabilità rimane presente se non viene implementato un adeguato controllo sulla lunghezza dei dati in ingresso. L'esercitazione evidenzia quindi l'importanza della validazione degli input e dell'utilizzo di funzioni che consentano di controllare la quantità di dati scritti in memoria.

Vincenzo Lembo